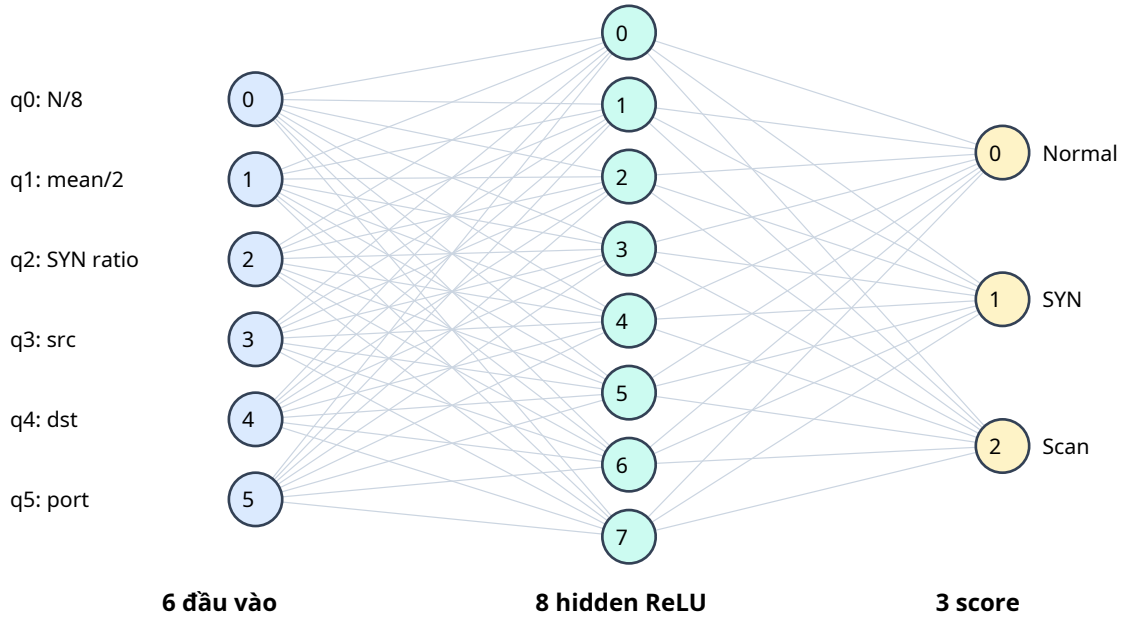
Học và hiểu phần AI V6 của bạn

Đề cương có giải thích, dữ liệu mẫu, thực hành và đáp án

Normal IoT-23 • SYN flood capture 34 • Vertical scan Chris Sanders



10 buổi học • 6 đặc trưng • MLP 6–8–3 • 83 tham số

Đối chiếu đúng model V6 và các file bàn giao. Có nêu nhãn xung đột, lỗi còn lại và giới hạn đánh giá. Ngày soạn: 08/11/2026.

Đọc PDF để học; mở bản HTML để thay sáu đặc trưng và xem phép tính. Các PCAP/CSV mẫu nằm trong gói ZIP đi kèm.

Mục lục

0. Cách học tài liệu này 3

1. Phần AI của bạn làm gì trong toàn hệ thống? 3

2. Kiến thức mạng đủ để đọc dữ liệu 4

3. Dataset thực sự của bạn gồm những gì? 5

4. Nhìn một file phải biết nó đang biểu diễn đơn vị nào 7

5. Làm sao biết dữ liệu là Normal, SYN flood hay scan? 8

6. Sáu đặc trưng: từ gói tin tới đầu vào 48 bit 9

7. Đọc và tính ba mẫu thật trong gói học 11

8. Model AI V6 hoạt động thế nào? 12

9. Model được train và chuyển sang FPGA thế nào? 14

10. Đọc báo cáo: đạt điều gì và còn thiếu gì? 16

11. Đọc core RTL và hiểu cách ghép 17

12. Đọc mã nguồn theo thứ tự nào? Chạy lệnh nào? 18

13. Lộ trình học mười buổi 20

14. Bài tập và đáp án 20

15. Chuẩn bị trình bày và câu hỏi bảo vệ 22

16. Từ điển ngắn và nguồn đối chiếu 24

Tài liệu học theo đúng model và dữ liệu đã bàn giao • Ngày 08/11/2026

0. Cách học tài liệu này

Mục tiêu là tự giải thích được đường đi từ một gói mạng đến kết quả AI, đọc được dữ liệu ba lớp, tính thử một lần suy luận và biết kết quả nào đã được kiểm chứng. Bạn không cần học thuộc toàn bộ Python hoặc Verilog trước khi bắt đầu.

Đây là đề cương có phần giải thích, ví dụ thật, thực hành và đáp án. Học lần lượt mục 1–8 để hiểu bản chất; mục 9–12 để hiểu huấn luyện và ghép nối; mục 13–16 để luyện trình bày. Mỗi buổi nên có một sản phẩm: bảng ghi chú, phép tính hoặc ảnh chụp kết quả thực hành.

Tài liệu dùng **model V6 của phần 2**, không lấy model V4 cũ để minh họa. SHA256 model là `3f1b2052ca30aad7f493b9f9091b86bcaf37dbf4c209363dea0c841f70f11515`. V6.1 của bản ghép trước có sửa đường MAC; trọng số AI vẫn là V6 này.

Trong gói học có ba PCAP/CSV mẫu dẫn xuất từ đúng các nguồn đã dùng. Chúng phục vụ học và đối chiếu, không bổ sung phiên tấn công độc lập hoặc thay đổi bộ trọng số. Bản HTML có bộ tính thử sáu đặc trưng và câu hỏi tự kiểm tra; PDF/DOCX dùng để đọc, in và ghi chú.

Sau khi học, bạn cần làm được sáu việc:

• Vẽ được vị trí phần AI trong hệ thống và nói rõ đầu vào/đầu ra.

• Phân biệt packet, flow, cửa sổ một giây và vector đặc trưng.

• Đọc bằng chứng để nhận xét Normal, SYN flood hoặc vertical scan; biết khi nào chưa đủ căn cứ.

• Từ counter tính raw, q, giá trị 48 bit, hidden và ba score.

• Giải thích cách train, chia dữ liệu, lượng tử hóa và kiểm model.

• Đọc báo cáo lỗi và bàn giao đúng hợp đồng giao tiếp cho người tích hợp.

1. Phần AI của bạn làm gì trong toàn hệ thống?

1.1. Hai đường đi phải phân biệt

Khi học và train trên máy tính:

```
PCAP + thông tin nhãn/nguồn
  -> lọc đúng hướng, IP đích và khoảng thời gian
  -> gom cửa sổ một giây
  -> sáu đặc trưng raw -> sáu q
  -> train trên máy tính -> model float -> model số nguyên
  -> xuất core RTL, header C và vector kiểm thử
```

Khi hệ thống hoạt động:

```
Gói Ethernet -> MAC/parser -> thống kê trước bộ lọc
  -> counter của một receiver trong một giây
  -> lượng tử hóa C -> core AI trên FPGA
  -> class_id + scores -> phần điều khiển/policy -> hành động
```

AI không mở file PCAP trên bo. PCAP là bản ghi để nghiên cứu và kiểm thử. Khi chạy thật, bộ trích đếm của hệ thống tạo các counter tương đương từ gói đang đi qua. CSV giúp người học nhìn được các giá trị đã trích.

1.2. Ranh giới công việc

Thành phần	Vai trò trong lần ghép này
Phần AI của bạn	Chuẩn sáu đặc trưng, xử lý dữ liệu, train/lượng tử hóa, model V6, core `ai_core`, helper C, kiểm số học, tài liệu giao tiếp
Hệ thống do người nhận tích hợp	CPU, bus, map thanh ghi, firmware hệ thống, nối/reset/clock của core, đọc kết quả và policy
Bộ trích đếm/lọc phần cứng	Cung cấp counter đúng chuẩn, gắn với receiver/cửa sổ; thuộc phần được phân công trích đếm của nhóm
MAC/PHY và bo	Thu/phát gói thực, chân tín hiệu và timing của hệ thống

Tên người phụ trách cụ thể cần theo phân công nhóm. Một file từng được đặt trong gói tích hợp không có nghĩa toàn bộ file đó thuộc phần AI của bạn.

Đầu vào của core AI: 48 bit chứa sáu q. **Đầu ra:** class 0/1/2 và ba score có dấu. Core không nhận IP nguồn, PCAP, nhãn hoặc chuỗi MQTT. Vì không nhận IP, core không tự biết phải chặn IP nào.

Câu tự kiểm: Nếu đổi sáu counter nhưng vẫn giữ IP cũ, model có thể đổi kết quả không? Có; model đọc đặc trưng, không đọc danh tính IP.

2. Kiến thức mạng đủ để đọc dữ liệu

2.1. Đọc một gói theo các lớp

Ethernet có MAC nguồn/đích. Bên trong IPv4 có IP nguồn/đích và IP Total Length. Bên trong TCP/UDP có cổng nguồn/đích. TCP có cờ điều khiển như SYN, ACK, RST, FIN. Một kết nối thường mô tả bằng protocol, IP nguồn, cổng nguồn, IP đích và cổng đích, gọi là 5-tuple.

Cổng đích cho biết dịch vụ được nhắm tới, không tự cho biết có tấn công. Cổng 1883 thường gắn với MQTT nhưng một gói gửi tới 1883 chưa chứng minh đó là phiên MQTT hoàn chỉnh. Cổng nguồn có thể thay đổi nhiều trong các kết nối hợp lệ.

2.2. Bắt tay TCP và “SYN mở”

```
Client -> Server : SYN
Server -> Client : SYN + ACK
Client -> Server : ACK
Sau đó mới trao đổi dữ liệu theo phiên TCP.
```

SYN là bước hợp lệ của TCP. Một SYN đơn lẻ không đủ để kết luận flood hoặc scan. Trong đặc trưng của bạn, SYN mở là gói TCP thỏa `(flags & 0x12) == 0x02`, nghĩa là bật SYN và không bật ACK. SYN-ACK bị loại khỏi tử số.

Mẫu số tỷ lệ SYN là **tất cả IPv4 hợp lệ tới receiver trong cửa sổ**, không chỉ các gói TCP. Tỷ lệ này mô tả hướng đang quan sát; nó không trực tiếp đo toàn bộ tỷ lệ bắt tay thất bại. Muốn kiểm bắt tay phải xem cả hướng trả lời.

2.3. Ba loại lưu lượng cần hiểu

Loại	Hành vi thường thấy	Không được kết luận chỉ từ
Normal	Giao tiếp phù hợp chức năng thiết bị, phiên/dịch vụ có ngữ cảnh hợp lệ; có thể có SYN, UDP, nhiều gói hoặc nhiều kết nối	Tốc độ thấp, không có SYN hoặc tên file “normal”
SYN flood	Dồn nhiều SYN mở nhằm gây quá tải/xáo trộn trạng thái kết nối; thường tập trung một/vài dịch vụ	Một SYN, một cửa sổ có tỷ lệ SYN cao hoặc một IP lạ
Vertical scan	Một nguồn thăm nhiều cổng đích trên cùng một host trong một khoảng thời gian; hay có chuỗi probe ngắn	Nhiều cổng nguồn hoặc tổng nhiều cổng do nhiều client hợp lệ tạo ra

Vertical scan khác horizontal scan: horizontal thường thăm cùng một/vài cổng trên nhiều host. Model hiện có ba lớp đã nêu, không có lớp horizontal scan riêng. Nó vẫn buộc chọn một trong ba class cho đầu vào hợp lệ, nên không coi class đó là bằng chứng model biết mọi loại tấn công.

SYN flood và scan đều có thể dùng SYN, cùng có gói nhỏ và tỷ lệ SYN cao. Điểm cần xem thêm là cổng đích, IP đích, nguồn, tốc độ và diễn biến thời gian. Với receiver /32, `dst_count` thường bằng 1 ở cả ba loại.

3. Dataset thực sự của bạn gồm những gì?

3.1. Nguồn gốc và nhãn

Nhóm	Nguồn đang dùng	Căn cứ nhãn
Normal, class 0	Các gói được lọc theo nhãn Benign gốc IoT-23; 28 tệp PCAP được chọn trong provenance	Nhãn gốc và quy trình ghép flow với packet; không lấy cả capture malware làm Normal
SYN flood, class 1	Đoạn đã rà soát của capture 34 IoT-23; receiver `71.61.66.148` trong khoảng đã ghi	Đoạn hành vi SYN flood đã kiểm chứng cho dự án; không đổi mọi DDoS thành SYN flood
Vertical scan, class 2	Mẫu `synscan.pcapng` công khai của Chris Sanders; chuyển sang PCAP để dùng	Nhóm rà soát hành vi quét; nhãn không phải nhãn ba lớp do tác giả gốc cung cấp

IoT-23 là bộ capture/nhật ký mạng IoT của Stratosphere Laboratory/CTU, có lưu lượng bình thường và nhiễm mã độc. Dự án chỉ dùng phần đã chọn/lọc, không có nghĩa đã dùng toàn bộ IoT-23 hoặc toàn bộ loại tấn công của bộ đó. Số 28 ở đây là số tệp Normal đã chọn, không phải số lớp hoặc số phiên tấn công độc lập.

Mẫu scan công khai có 2.011 frame tổng cộng. Hướng scanner `172.16.0.8` tới `64.13.134.52` có 1.994 SYN mở, nhắm 1.000 cổng đích phân biệt; 17 frame còn lại là hướng trả lời. **1.994 gói không đồng nghĩa 1.994 cổng**, vì một cổng có thể bị thăm nhiều lần. Đây là một đợt scan, không phải 1.994 phiên kiểm thử độc lập.

3.2. Các biến thể mô phỏng V6

Ngoài cache gốc, V6 dùng 120 profile từ đúng các nguồn trên. Mã chuẩn bị dữ liệu đổi khoảng cách thời gian gói tấn công và thêm nền từ Normal34. Byte gói tấn công được giữ; gói nền có thể đổi IP/cổng để ghép vào cùng receiver rồi tính lại checksum.

• SYN train lấy 12.000 gói đầu; development lấy 12.000 gói tiếp theo của cùng đợt.

• Scan dùng cùng nguồn với các hệ số thời gian khác nhau.

• Normal mô phỏng dùng mẫu SYN có SYN-ACK tương ứng và mẫu ACK từ Normal34, bố trí ở các mức tải có kiểm soát.

• Tên `sensor7` mô tả cách trộn nền; không chứng minh payload là dữ liệu cảm biến MQTT mới thu.

Các profile giúp model gặp nhiều cách trộn và mức tải. Chúng **không tạo nguồn tấn công độc lập mới**. Chia một capture thành nhiều file, đổi timestamp hoặc lặp mẫu không làm nguồn đó trở thành nhiều phiên thu mạng độc lập.

3.3. Vì sao có SQLite cache?

Cache giữ các q trùng nhau dưới dạng `(label, packed, count)`. Ví dụ count = 10.000 nghĩa là có 10.000 cửa sổ mang q/nhãn đó trong dữ liệu đã tổng hợp; không phải 10.000 dòng fit khác biệt. Cache giúp tránh đọc lại hàng chục GB PCAP và hàng triệu dòng CSV.

`native_patterns.sqlite` giữ thống kê gốc; `v5_patterns.sqlite` còn giữ các biến thể scan đã dùng ở V5. Không thể dùng cache để khôi phục chính xác timestamp, IP hoặc từng gói PCAP đã mất. Muốn điều tra lỗi theo gói cần PCAP và thông tin nguồn/cửa sổ tương ứng.

Một ví dụ trong cache gốc: q = [0,20,127,1,1,1] mang nhãn Normal xuất hiện 21.950.632 lần. Điều này cho thấy q2 = 127 không tự là nhãn tấn công. Vì q0 = 0 nhưng có nguồn/đích/cổng, đây là các cửa sổ không rỗng với N trong khoảng 1–7 trước lượng tử hóa. Nhãn Native Benign vẫn cần được đọc trong đúng ngữ cảnh nguồn, không suy ra chỉ từ tỷ lệ SYN.

Câu tự kiểm: Model thấy 39 triệu cửa sổ Normal có đồng nghĩa có 39 triệu mẫu Normal khác nhau không? Không; nhiều cửa sổ trùng sáu q và được gom bằng count.

4. Nhìn một file phải biết nó đang biểu diễn đơn vị nào

Dạng file	Một phần tử là gì?	Đọc để làm gì?
PCAP/PCAPNG	Một packet/frame, có timestamp và bytes	Xem IP/cổng/cờ, thời gian, hướng đi và phản hồi bằng Wireshark
CSV flow/Zeek	Một flow hoặc bản ghi kết nối; có thể bao nhiều giây	Xem nhãn gốc và quan hệ hai chiều; phục vụ lọc
CSV đặc trưng	Một receiver trong một cửa sổ một giây	Đầu vào học: sáu raw, sáu q và metadata
model.json	Trọng số và cấu hình model	Xem cấu trúc số học của model số nguyên
metrics.json	Thống kê trên các nhóm đánh giá	Đọc ma trận nhầm lẫn và lỗi còn lại
vectors.txt	Một đầu vào q, score đóng gói và class mong đợi	So Python với RTL, không phải capture mới

`.gz` nghĩa là nén gzip, không phải một loại nhãn. CSV flow có thể có `orig_pkts`, `resp_pkts`, `id.orig_h`, `id.resp_h`, `duration`. CSV đặc trưng có `packet_count`, `mean_ip_bytes`, `syn_ratio_q127`, `src_count`, `dst_count`, `dst_port_count`.

Không đổi tên CSV flow thành "features.csv" rồi đưa vào model. Một flow 30 giây không phải một cửa sổ một giây; 10.000 flow không đồng nghĩa 10.000 packet. Chiều originator/responder của Zeek cũng không tự tương đương chiều ingress tới receiver mà core cần.

Trong CSV mô phỏng V6 còn có metadata:

Cột	Ý nghĩa
`profile`, `split`	Profile nào; train hoặc development
`label`	Nhãn kịch bản, 0/1/2; không phải kết quả dự đoán
`window_start`	Mốc bắt đầu cửa sổ theo giây timestamp
`status`	Hàng đủ điều kiện hay cần ngữ cảnh/xung đột
`attack_count`	Số gói được đánh dấu là thành phần tấn công trong mô phỏng
`attack_port_count`	Số cổng đích phân biệt của riêng thành phần tấn công
`background_count`	Số gói nền được trộn
`q0` đến `q5`	Sáu giá trị sau lượng tử hóa

Các cột `label`, `status`, `attack_count`, `profile` không được đưa làm đầu vào AI. Nếu cho model thấy nhãn hoặc dấu vết tên nguồn, kết quả có thể đẹp nhưng bị rò rỉ thông tin.

5. Làm sao biết dữ liệu là Normal, SYN flood hay scan?

5.1. Luôn tách ba câu hỏi

Nhãn đang ghi là gì? Đọc `label`, nhãn flow gốc và provenance. Đây là câu trả lời theo bộ dữ liệu/kịch bản.

Gói tin thể hiện hành vi gì? Kiểm IP/cổng/cờ, tốc độ, hướng trả lời, số cổng theo từng nguồn và diễn biến nhiều cửa sổ. Đây là bằng chứng mạng.

Model dự đoán gì? Tính q rồi chạy model. Dự đoán có thể khác nhãn hoặc không đủ để giải thích hành vi. Không sửa nhãn cho khớp model.

Không có mẹo nhìn một dòng CSV bất kỳ và luôn biết chắc loại lưu lượng. Khi sáu đặc trưng giống hệt nhau giữa hai nhãn, riêng model hiện tại không có thông tin để phân biệt. Phải trở lại ngữ cảnh gói/phiên hoặc bổ sung đặc trưng trong một phiên bản nghiên cứu mới.

5.2. Quy trình đọc một mẫu

1. Xác định file thuộc PCAP, flow CSV hay feature CSV; xác định nhãn và căn cứ nhãn.

2. Chọn đúng receiver /32, khoảng thời gian và hướng ingress. Ghi rõ đang xem một giây hay cả capture.

3. Với TCP, xem SYN không ACK, cổng **đích**, tỷ lệ SYN và phản hồi ở hướng ngược lại.

4. Gom theo **từng IP nguồn**: nguồn nào thăm những cổng đích nào trên cùng host? Nếu chỉ có tổng nhiều cổng của nhiều nguồn thì chưa đủ kết luận vertical scan của một nguồn.

5. Đối chiếu dòng đặc trưng, status và dự đoán. Ghi "phù hợp với...", "chưa đủ bằng chứng" hoặc "cần rà soát nhãn" khi cần.

5.3. Thực hành Wireshark bằng các file trong gói

Giải nén ZIP học. Các PCAP trong `vi_du/` còn nén gzip. Trên Ubuntu, trong thư mục gói chạy:

```
python3 HOC_AI.py giai-nen-pcap
```

Lệnh tạo bản `.pcap` trong `vi_du/pcap_da_giai_nen/`, giữ nguyên `.pcap.gz`. Mở Wireshark File Open chọn một PCAP đó. Đầu tiên xem toàn bộ file, chưa lọc, để nhận biết hai chiều nếu có. Thêm cột `ip.src`, `ip.dst`, `tcp.dstport`, `tcp.flags` khi cần.

Muốn xem	Display filter
Hướng tới receiver SYN/Normal mẫu	`ip.dst == 71.61.66.148`
SYN mở	`tcp.flags.syn == 1 && tcp.flags.ack == 0`
SYN-ACK	`tcp.flags.syn == 1 && tcp.flags.ack == 1`
Scanner gốc tới đích scan	`ip.src == 172.16.0.8 && ip.dst == 64.13.134.52`
Hướng tới đích scan	`ip.dst == 64.13.134.52`
Cửa sổ được minh họa	`frame.time_epoch >= 1800000003 && frame.time_epoch < 1800000004`

Nối hai điều kiện bằng &&, ví dụ lọc receiver rồi đúng cửa sổ. Số "Displayed" lúc đó là số frame qua display filter, không tự là số flow hoặc số hàng CSV.

Trong Statistics I/O Graphs, dùng khoảng một giây để nhìn số gói theo thời gian. Trong Statistics Conversations, xem các cặp IP/TCP, rồi lọc một nguồn để kiểm cổng đích. Endpoints/conversations của **cả file** không tự là cardinality của một cửa sổ; phải áp đúng bộ lọc và phạm vi.

Ba PCAP mô phỏng đi kèm chỉ chứa các gói gửi tới receiver: không đủ hướng trả lời để tự chứng minh lại toàn bộ bắt tay TCP. Bằng chứng SYN-ACK của mẫu nền nằm trong nguồn Normal34 gốc. Không được kết luận "không thấy SYN-ACK trong file một chiều nên chắc chắn là tấn công".

Mốc `1800000003` là timestamp đã bố trí khi tạo mô phỏng, không phải ngày thu IoT-23 gốc. Wireshark có thể hiển thị giờ theo múi giờ; dùng `frame.time_epoch` để đối chiếu chính xác.

5.4. Đọc status đúng cách

• `eligible`: được cho phép dùng trong bước fit/selection theo quy tắc chuẩn bị dữ liệu, không có nghĩa model chắc chắn nhận đúng.

• `sparse_attack_needs_context`: cửa sổ SYN dẫn xuất có dưới 20 gói tấn công; giữ để xem, không tự đổi thành Normal.

• `sparse_scan_needs_context`: cửa sổ scan dẫn xuất có dưới ba cổng tấn công phân biệt; có thể chỉ còn nền trong cửa sổ đó.

• `native_label_alias_ambiguous`: q của cửa sổ dẫn xuất trùng q mang nhãn gốc khác; giữ hàng và báo riêng, loại khỏi fit/selection.

20 gói và ba cổng là quy tắc xử lý các profile của dự án, không phải ngưỡng mạng phổ quát hay câu lệnh quyết định của MLP. Nhãn kịch bản 2 có thể còn trên một hàng không có gói scan; status giúp tránh coi hàng đó là bằng chứng scan rõ ràng.

6. Sáu đặc trưng: từ gói tin tới đầu vào 48 bit

6.1. Phạm vi thống kê

Mỗi mẫu ứng với **một receiver IPv4 /32 trong một giây**, cùng quy ước mốc cửa sổ. Dùng các gói IPv4 hợp lệ tới receiver; loại phân mảnh và gói thiếu bytes so với IP Total Length. ARP và hướng ngược lại không được cộng. Thống kê cần đặt trước bộ lọc để vẫn thấy nguồn đã chặn.

Một receiver /32 nghĩa là một địa chỉ IP chính xác, không phải cả mạng /24. Cùng PCAP nhưng gom cả /24 hoặc cả hai hướng có thể tạo vector khác và làm sai ý nghĩa model.

6.2. Bảng công thức phải nắm

Gọi N là số gói, B là tổng IPv4 Total Length, S là số SYN mở. Các phép chia đều lấy phần nguyên xuống.

Chỉ số	Tên trong CSV	raw	Divisor	q đưa vào core
0	packet_count	N	8	min(127, floor(N/8))
1	mean_ip_bytes	floor(B/N)	2	min(127, floor(raw1/2))

Chỉ số	Tên trong CSV	raw	Divisor	q đưa vào core
2	syn_ratio_q127	floor(127×S/N)	1	raw2, bão hòa tối đa 127
3	src_count	Số IP nguồn phân biệt	1	min(127, raw3)
4	dst_count	Số IP đích phân biệt	1	min(127, raw4)
5	dst_port_count	Số cổng đích TCP/UDP phân biệt	1	min(127, raw5)

`mean_ip_bytes` dùng **IP Total Length**, không dùng độ dài frame Ethernet hoặc số byte payload MQTT. Cổng của ICMP không được giả thành một cổng TCP/UDP. `src_count` là đếm số địa chỉ khác nhau, không phải giá trị số của một địa chỉ.

Đếm phân biệt của bản V6 này là exact. Một bitmap/hash có va chạm có thể giảm số đếm và đổi kết quả. Core không tự sửa sai khác do bộ trích đếm.

6.3. raw, q và counter không phải cùng một định dạng

CSV sáu raw chứa **trung bình byte** và **tỷ lệ SYN đã tính**. Hàm C `ai_v6_prepare_window` lại nhận **tổng byte B và số SYN S thực** để tự tính hai giá trị đó. Truyền `mean_ip_bytes` vào vị trí B hoặc phần trăm UART vào vị trí S sẽ sai.

```
Thứ tự counter cho C:
[N, B, S, src_count, dst_count, dst_port_count]

Thứ tự raw cho model/CSV:
[N, floor(B/N), floor(127*S/N), src_count, dst_count, dst_port_count]
```

N = 0: bỏ suy luận, không phát START và không dùng score cũ để quyết định. N từ 1 đến 7 vẫn hợp lệ dù q0 bằng 0. Như vậy `q0 == 0` không phải điều kiện cửa sổ rỗng.

6.4. Lượng tử hóa làm mất thông tin gì?

N = 8 và N = 15 đều cho q0 = 1. N từ 1.016 trở lên đều cho q0 = 127. Có 127 hay 1.000 cổng thì q5 đều bằng 127. Hai mean 40 và 41 đều cho q1 = 20. Model chỉ nhìn q nên không thể phân biệt các giá trị đã nhập làm một.

Đây là đánh đổi để số học nhỏ, nhất quán giữa Python/C/RTL. Không có nghĩa 7 bit luôn đủ cho mọi loại mạng. Nếu thay divisor, cửa sổ hoặc cách đếm thì phải đánh giá lại model và kiểm toàn đường xử lý.

6.5. Đóng gói 48 bit

```
bit 47..40 | 39..32 | 31..24 | 23..16 | 15..8 | 7..0
    q5     |   q4   |   q3   |   q2   |  q1   |  q0
```

`feat = q0 + (q1<<8) + (q2<<16) + (q3<<24) + (q4<<32) + (q5<<40).`

Đọc chuỗi hex từ trái sang phải sẽ thấy q5 trước q0. Đây là quy ước vị trí bit, không phải tự suy ra cách truyền bytes qua UART hoặc bus. Phần adapter cần tuân thủ đúng low32/high16 đã định nghĩa.

7. Đọc và tính ba mẫu thật trong gói học

Cả ba ví dụ dưới đây lấy từ dòng có `window_start = 1800000003`. File CSV giữ nguyên nội dung sau giải nén; PCAP được sao nguyên từ profile đã dùng. Các counter đã được trích lại từ PCAP để đối chiếu.

7.1. Normal: normal_train_14_sensor7

Một cửa sổ có N = 14, B = 768 byte IPv4, S = 2, một IP nguồn, một IP đích, một cổng đích. Metadata: label 0, status eligible, attack_count 0, background_count 14.

```
raw = [14, 54, 18, 1, 1, 1]
q   = [ 1, 27, 18, 1, 1, 1]
feat = 0x010101121b01
scores = [679, -411, -223] -> class 0: Normal
```

Tính tay: floor(768/14) = 54; floor(127×2/14) = 18; floor(14/8) = 1; floor(54/2) = 27. Dữ liệu Normal vẫn có SYN: hai trong 14 gói. Nhìn SYN rồi gọi flood sẽ gán nhãn sai.

Cả PCAP có 420 frame; CSV có 29 cửa sổ hoàn chỉnh. Hai con số đếm đơn vị khác nhau. Tên có `sensor7` nhưng tốc độ Normal profile này là 14 gói/giây; phải đọc hàng CSV và manifest thay vì đoán từ một phần tên file.

7.2. SYN flood: syn_train_512_sensor7

Một cửa sổ có N = 489, B = 19.728, S = 483, hai nguồn, một đích, một cổng đích. Metadata: label 1, status eligible, attack_count 482, background_count 7.

```
raw = [489, 40, 125, 2, 1, 1]
q   = [ 61, 20, 125, 2, 1, 1]
feat = 0x0101027d143d
scores = [718, 1142, -2011] -> class 1: SYN flood
```

S = 483 lớn hơn attack_count 482 vì bảy gói nền cũng có một SYN hợp lệ. Tỷ lệ SYN raw là 125 trên thang 127, không phải 125%. Nguồn thứ hai là nền, nên hai nguồn không tự chứng minh tấn công phân tán.

512 trong tên là tốc độ mục tiêu theo cách co giãn thời gian profile; không bắt buộc mỗi cửa sổ có đúng 512 gói tấn công. Cách bố trí giữ cấu trúc thời gian tương đối của đoạn nguồn nên số gói từng giây có thể thay đổi.

Cả PCAP có 12.164 frame; CSV có 22 cửa sổ hoàn chỉnh. Cửa sổ ở biên không đầy đủ được bỏ khỏi CSV học, không phải tự ý xóa nhãn hoặc làm mất gói trong PCAP.

7.3. Vertical scan: scan_train_2_one_syn

Một cửa sổ có N = 112, B = 4.944, S = 112, hai nguồn, một đích, 70 cổng đích. Metadata: label 2, status eligible, attack_count 111, attack_port_count 69, background_count 1.

```
raw = [112, 44, 127, 2, 1, 70]
q   = [ 14, 22, 127, 2, 1, 70]
feat = 0x4601027f160e
scores = [-4824, -1895, 7896] -> class 2: Vertical scan
```

Tỷ lệ SYN cũng cao như flood, nhưng có nhiều cổng đích. Riêng scanner thăm 69 cổng trong cửa sổ; nền thêm cổng dịch vụ khác nên tổng là 70. Khi nhìn PCAP cần lọc nguồn scanner để không trộn hành vi của hai nguồn.

Số 2 trong tên là **hệ số giãn thời gian**, không phải hai gói/giây. Class 2 của scan là một quy ước khác, không lấy từ con số trong tên file. Cả PCAP mô phỏng có 2.004 frame (1.994 scanner + 10 nền) và CSV có chín cửa sổ hoàn chỉnh; khác bản nguồn 2.011 frame có cả phản hồi.

7.4. Bảng so sánh một cửa sổ

Đại lượng	Normal mẫu	SYN mẫu	Scan mẫu
Nhãn	0	1	2
N	14	489	112
SYN mở thực	2	483	112
Tỷ lệ thang 127	18	125	127
IP nguồn phân biệt	1	2	2
IP đích phân biệt	1	1	1
Cổng đích phân biệt	1	1	70
Dự đoán V6	0	1	2

Đây là ba ví dụ cụ thể, không phải bảng ngưỡng phổ quát. Model không viết sẵn luật “trên 100 gói là flood” hay “trên 10 cổng là scan”.

8. Model AI V6 hoạt động thế nào?

8.1. Cấu trúc 6–8–3

Model là MLP, mạng nơ-ron truyền thẳng với một lớp ẩn ReLU:

```
6 đầu vào q -> 8 neuron hidden -> 3 score đầu ra
                                [Normal, SYN flood, Vertical scan]
```

Có thể gọi là ba tầng biểu diễn nếu tính cả input, hoặc hai lớp có trọng số. Cách nói chính xác khi bảo vệ: “sáu đầu vào, một lớp ẩn tám neuron ReLU, ba đầu ra”.

Tên trong model.json	Kích thước	Ý nghĩa
w1	8 × 6	Mỗi hàng là sáu trọng số đi vào một neuron ẩn
b1	8	Một bias cho mỗi neuron ẩn
w2	3 × 8	Mỗi hàng là tám trọng số tạo một score
b2	3	Một bias cho mỗi score
divisors	6	Chuyển raw thành q
shift1, shift2	5 và 0	Dịch số học sau tổng hidden/output

Số tham số học: 6×8 + 8 + 8×3 + 3 = **83**, gồm 72 weight và 11 bias. Sáu q có 42 bit dữ liệu hữu ích nhưng được xếp trong sáu byte thành 48 bit để nối giao diện thuận tiện. Input không phải sáu neuron có thêm weight riêng. Số neuron ẩn không phải số lớp dữ liệu.

Thiết kế nhỏ phù hợp mục tiêu core FPGA dùng tài nguyên ít. Không có kết quả tìm kiếm kiến trúc để khẳng định tám neuron là tối ưu cho mọi mạng; đây là kiến trúc được chọn và kiểm trong dự án.

8.2. Một neuron học gì?

Neuron là một đơn vị tính: nhận nhiều số, nhân hệ số rồi cộng lại. Weight là hệ số được học cho từng đầu vào; bias là số cộng độc lập với đầu vào, giúp dịch mức đáp ứng của neuron. Chúng được học từ dữ liệu, không phải nhãn 0/1/2.

Một neuron cộng nhiều đặc trưng có trọng số rồi thêm bias. Weight dương/âm làm tổng tăng/giảm theo đầu vào, nhưng không được diễn giải một weight đơn lẻ thành quy luật của cả model: các neuron còn tương tác qua lớp sau, ReLU và bão hòa.

Hidden là biểu diễn trung gian, không có quy ước h0 là flood hay h1 là scan. Tám neuron cùng tạo cơ sở để ba output phân loại.

ReLU là max(0, x). Không có ReLU, nhiều lớp tuyến tính nối tiếp vẫn chỉ tương đương một phép biến đổi tuyến tính. Trong bản số nguyên, hidden còn bão hòa ở 127.

8.3. Công thức suy luận số nguyên đúng bản V6

```
a[j] = b1[j] + sum(w1[j][i] * q[i]), i = 0..5
h[j] = clip(a[j] >> 5, 0, 127), j = 0..7

u[k] = b2[k] + sum(w2[k][j] * h[j]), j = 0..7
s[k] = clip(u[k] >> 0, -32768, 32767), k = 0..2

class_id = chỉ số score lớn nhất; hòa thì chọn chỉ số nhỏ nhất
```

Weight là int8 có dấu; bias và accumulator là int32. Hidden ở 0..127, output score int16 có dấu. >> ở đây là dịch số học có dấu, khác dịch logic; với số âm không nên thay tùy tiện bằng phép chia C có quy tắc làm tròn khác.

Core trả ba **score**, không chạy softmax và không trả phần trăm tin cậy. Score âm vẫn có thể thắng nếu hai score còn lại âm hơn. `score_scale = 64` là metadata của bước chuyển model; không phải confidence 64%, divisor đầu vào hoặc phép nhân thêm phải làm khi nhận score.

8.4. Tính tay từ đúng ví dụ SYN

Với q = [61, 20, 125, 2, 1, 1], hàng đầu của w1 là [-25, 3, 40, -127, 2, -35], b1[0] = -1135:

```
a[0] = -25*61 + 3*20 + 40*125 - 127*2 + 2*1 - 35*1 - 1135
     = 2113
h[0] = clip(2113 >> 5, 0, 127) = 66

Tính đủ tám neuron:
h = [66, 0, 11, 97, 0, 0, 1, 0]

score0 = 9*66 - 91*11 + 6*97 + 4*1 + 539 = 718
score1 = -6*66 + 15*11 + 16*97 - 6*1 - 173 = 1142
score2 = -2*66 + 88*11 - 26*97 + 3*1 - 328 = -2011
```

Score1 lớn nhất nên class 1. Đây là suy luận: dùng weight đã học để tính, không có bước học lại tại đây.

8.5. Một lỗi hiện có rất đáng học

Cửa sổ development thật trong báo cáo có 803 gói SYN, mean 40, tỷ lệ 127, một nguồn, một đích, một cổng:

```
raw = [803, 40, 127, 1, 1, 1]
q   = [100, 20, 127, 1, 1, 1]
scores V6 = [1413, 1413, -3174]
class = 0, vì score0 và score1 hòa, ưu tiên chỉ số nhỏ hơn
label = 1 -> đây là một SYN bị bỏ sót
```

Không được nói "mọi SYN tải cao đều nhận đúng". Cũng không tự đổi quy tắc hòa chỉ để sửa ví dụ này: thay đổi đó có thể tạo lỗi nơi khác và làm lệch chuẩn Python/C/RTL. Muốn sửa phải tạo ứng viên mới rồi đánh giá lại.

9. Model được train và chuyển sang FPGA thế nào?

9.1. Train khác inference

Train chạy trên máy tính, dùng q và nhãn để điều chỉnh weight/bias. Inference dùng weight/bias đã chốt để phân loại q mới. Core FPGA của bạn chỉ inference; nó không tự học từ gói mới và không cần nhãn khi chạy.

Python được dùng chuẩn bị dữ liệu, huấn luyện và tạo kết quả tham chiếu. C chuẩn bị đầu vào đúng quy ước. Verilog triển khai phép tính số nguyên. Không phải copy một chương trình Python vào FPGA rồi bo tự chạy scikit-learn.

9.2. Chuỗi huấn luyện hiện tại

1. Kiểm checksum cache/manifest/CSV để biết đang dùng đúng dữ liệu.

2. Đọc q gốc và số lần xuất hiện; thêm các cửa sổ dẫn xuất có status eligible vào train/development tương ứng.

3. Gom q trùng theo lớp. Số q khác nhau theo lớp trong fit là [1.549, 395, 329].

4. Cân bằng số điểm fit: mỗi lớp có 1.549 hàng bằng cách lặp q của lớp ít; tổng 4.647 hàng. Shuffle seed 23 để tái tạo. Việc lặp không tạo lưu lượng độc lập mới.

5. Train MLP float với q/128, hidden (8,), ReLU, solver L-BFGS, alpha 0,01, random_state 5, max_iter 2.000 cho ứng viên đã chọn.

6. Chuyển float sang số nguyên và chọn cấu hình lượng tử hóa bằng dữ liệu train; kiểm giới hạn int8/int32.

7. Nếu chạy search đầy đủ, so 27 ứng viên theo development cùng nguồn và cổng hồi quy trên thống kê gốc; chọn ứng viên đáp ứng tiêu chí.

8. Xuất model, tệp hex, RTL/header/vector và lặp bài kiểm trước khi thay bản đã chốt.

Alpha là mức regularization của bước fit, không phải learning rate của core. `max_iter` là giới hạn vòng của bộ tối ưu L-BFGS, không nên gọi là chính xác 2.000 epoch đã chạy. Seed giúp tái tạo nhưng không bảo đảm giống bytes giữa mọi phiên bản thư viện/máy.

Trong train, nhãn cho biết đầu ra mong muốn; mạng dự đoán, hàm mất mát log-loss đánh giá mức sai, gradient/backpropagation cho biết cách thay đổi weight/bias. L-BFGS dùng thông tin gradient để tối ưu tham số. Không phải người viết gán tay weight theo tên tấn công. Với mạng nhỏ và dữ liệu q đã

gom, không cần GPU để dùng gói này. Cân bằng điểm fit giúp lớp ít không bị lấn hoàn toàn, nhưng không tạo đa dạng hành vi tấn công mới.

9.3. Có hai phép scale khác nhau

Scale đặc trưng: raw chia divisors [8,2,1,1,1,1] và clip để ra q. Model float được fit trên q/128.

Chuyển mạng sang số nguyên: weight float được chuyển bằng scale từng neuron ẩn và scale output; làm tròn bằng round-to-nearest-even, chọn shift chung, ép weight int8. Có 40 lựa chọn lượng tử hóa cho mỗi ứng viên float trong quy trình đầy đủ, gồm cấu hình nền và lưới rescale. Sau làm tròn, mạng có thể khác float nên cần kiểm số học và đánh giá lại.

Khi so hai file, chú ý `float_model.json` lưu w1 theo chiều 6×8 và w2 theo chiều 8×3 của scikit-learn; model số nguyên lưu hàng theo neuron nên là 8×6 và 3×8. Còn có rescale/làm tròn, nên không đối chiếu bằng cách đòi các số cùng vị trí hai file giống nhau.

Trong model số nguyên đã chốt, các scale này đã được hấp thụ vào weight/bias. Không chia q thêm 128 trong core hoặc áp `hidden_scales` lần thứ hai. Core thực hiện đúng công thức số nguyên ở mục 8.

9.4. Chia dữ liệu và rò rỉ

Train dùng học tham số. Development dùng so sánh/lựa chọn ứng viên. Test độc lập phải giữ ngoài quá trình đó, ưu tiên các phiên thu mới, ngày khác, thiết bị khác hoặc cấu hình khác và có nhãn kiểm được.

Không chia ngẫu nhiên các giây liền nhau của cùng một đợt rồi gọi đó là chứng minh nhận diện trên nguồn độc lập. Gói V6 có development khác tốc độ/thời gian nhưng chung capture cha. Lab lịch sử từng được xem qua cũng không trở lại thành test mới chỉ vì đổi tên file.

Các dạng rò rỉ cần nhận ra: cùng packet/q đi cả train lẫn test; cùng nguồn đã điều chỉnh theo kết quả test; lấy nhãn, tên profile hoặc địa chỉ cố định làm feature; fit scaler trên cả train/test. Bộ hiện tại không dùng IP hay tên file làm sáu đầu vào, nhưng vẫn có giới hạn chung nguồn đã được báo rõ.

9.5. Tái tạo đúng bản đã chốt

Gói `phan2_ai_v6_train_va_du_lieu.zip` có `train.py`, `prepare_data.py`, runtime, cache và dữ liệu. Môi trường ghi nhận: Python 3.12.14, NumPy 2.2.6, SciPy 1.18.1, scikit-learn 1.6.1, Scapy 2.6.1, threadpoolctl 3.7.0.

Lệnh mặc định alpha 0,01/seed 5 đã tái tạo đúng SHA256 V6 trong môi trường đó. Hash xác nhận đồng nhất file, không chứng minh model tổng quát tốt. Nếu train ra hash khác, giữ thành ứng viên riêng; không ghi đè gói đã bàn giao để ép “giống bản mới”.

10. Đọc báo cáo: đạt điều gì và còn thiếu gì?

10.1. Đọc confusion matrix

Trong báo cáo này **hàng là nhãn gốc, cột là dự đoán**, thứ tự Normal/SYN/scan. Trên thống kê gốc:

Nhãn gốc / Dự đoán	Normal	SYN flood	Vertical scan
Normal	39.253.923	12	0
SYN flood	0	21	0
Vertical scan	0	0	4

Có 12 Normal bị gọi là SYN. Với lớp SYN: TP = 21, FP = 12, FN = 0. Một model luôn gọi Normal cũng đạt accuracy rất gần 100% trên tập lệch lớp này dù bỏ lỡ mọi tấn công. Vì thế không chỉ báo accuracy.

```
Accuracy = tổng đường chéo / tổng số mẫu
Precision của lớp SYN = TP / (TP + FP) = 21/33 = 63,64%
Recall của lớp SYN = TP / (TP + FN) = 21/21 = 100%
F1 = 2 * precision * recall / (precision + recall)
Macro-F1 = trung bình F1 của ba lớp, mỗi lớp có trọng số như nhau
```

Accuracy gốc khoảng 99,99996943%; macro-F1 khoảng 92,59%. Đây là độ khớp trên dữ liệu gốc đã tham gia quá trình phát triển, không phải tỷ lệ phát hiện được xác nhận trên mạng thực mới.

10.2. Các nhóm kết quả khác nhau

Nhóm	Kết quả cần nhớ	Cách nói đúng
Gốc	Normal sai 12; SYN 21/21; scan 4/4	Thống kê trên cửa sổ gốc, không đếm thành số phiên tấn công
Development eligible cùng nguồn	Normal 348/348, SYN 2.355/2.356, scan 640/640	Có một SYN 803 gói bị bỏ sót; không phải test độc lập
Lab lịch sử	218/219, một Normal nhận scan	Hồi quy trên dữ liệu lab cũ
RTL	3.762 vector khớp score/class, 84 chu kỳ	Kiểm phép tính và giao thức core
C packing	41 trường hợp, 40 hợp lệ và một rỗng	Kiểm chuyển counter thành feat/validity

Các hàng sparse/ambiguous không được tự cộng vào số eligible đúng. Trong `metrics.json` có phần báo mọi status riêng để người đọc không bỏ qua những hàng khó.

10.3. Xung đột do cùng q

q = [127,20,127,1,1,1] có các cửa sổ nhãn Normal và SYN trong nguồn gốc. Model xác định chỉ từ q nên cùng q phải ra cùng class. Nếu ra SYN, các Normal đó sai; nếu ra Normal, các SYN tương ứng sai.

Có 11 Normal thuộc kiểu trùng q này trong các lỗi gốc. Không tự kết luận nhãn gốc sai; phải xem lại quy trình lọc, lưu lượng và cách lượng tử hóa. Các hướng nghiên cứu tiếp gồm kiểm nhãn, thêm ngữ cảnh phiên, thay đặc trưng hoặc thay độ phân giải trong một phiên bản có đánh giá lại.

10.4. Bốn loại kiểm chứng không thay nhau

Đúng dữ liệu/nhãn: đã lấy đúng nguồn và phép trích đúng hay chưa?

Đúng số học: Python, C packing và RTL có khớp đầu vào/score/class hay chưa?

Đúng triển khai phần cứng: synthesis, P&R, timing, reset, CDC và giao tiếp bo có đạt hay chưa?

Hiệu quả phát hiện: trên các phiên mới có giữ được precision/recall và mức báo nhầm chấp nhận được hay chưa?

PASS RTL không chứng minh accuracy mạng mới. Timing đạt không chứng minh nhãn đúng. Gói AI riêng không có kết quả chạy bo thật mới; báo cáo top tích hợp trước thuộc phạm vi top đó.

11. Đọc core RTL và hiểu cách ghép

11.1. Vì sao ít tài nguyên mà tính nhiều neuron?

Core dùng một bộ nhân–cộng chia sẻ theo thời gian. Nó lần lượt tính 8×6 = 48 tích cho hidden rồi 3×8 = 24 tích cho output, tổng 72 tích. Nó không cần 72 bộ nhân chạy đồng thời.

Các trạng thái trong RTL:

State	Việc làm
IDLE	Chờ START, chốt toàn bộ feat
HSET	Nạp bias cho một neuron ẩn
HMAC	Sáu bước nhân–cộng; dịch/ReLU/clip và lưu hidden
OSET	Nạp bias cho một output
OMAC	Tám bước nhân–cộng; clip và lưu score
FINISH	Chốt scores/class, phát DONE và trở lại IDLE

Từ cạnh START được chấp nhận đến DONE là 8×(1+6) + 3×(1+8) + 1 = **84 chu kỳ** theo testbench. Đây là độ trễ core, không gồm cửa sổ thống kê, bus hoặc thời gian chặn. Ở clock 27 MHz, phép chia lý thuyết 84/27 MHz khoảng 3,11 microsecond; không phải số đo độ trễ toàn hệ thống trên bo.

Dù core tính trong microsecond, hệ thống dùng đặc trưng một giây vẫn phải tích lũy cửa sổ. Không nói hệ thống phát hiện ngay một gói tấn công sau 3,11 microsecond.

11.2. Handshake và kiểu số

• Reset `rst_n` mức 0, **đồng bộ cạnh lên clk**; giữ qua ít nhất một cạnh lên.

• Khi busy = 0, giữ feat ổn định và phát START một chu kỳ. Core chốt feat tại cạnh nhận.

• START khi busy bị bỏ qua, không tự xếp hàng giao dịch.

• DONE là xung một chu kỳ, báo kết quả mới. Scores được giữ sau đó nhưng không có nghĩa mỗi lần đọc là một dự đoán mới.

• Score0 ở bit 15:0, score1 ở 31:16, score2 ở 47:32; đọc từng lane như signed int16.

Nếu giữ START lên mức 1 liên tục, core có thể nhận lại khi rảnh; người tích hợp cần xung đúng chuẩn. Khi N = 0 phải bỏ START và bỏ quyết định dựa trên cửa sổ đó, không đọc lại class cũ.

11.3. CPU và AI trao đổi gì?

Hàm C nhận counter, tạo một uint64 chứa 48 bit có nghĩa. Phần adapter của người nhận viết low32/high16 vào thanh ghi, phát START, chờ DONE và đọc class/scores. Map địa chỉ thanh ghi thuộc bus wrapper của họ, không nằm trong core AI riêng.

Cùng class 1 nhưng cần biết nguồn nào để xử lý. Core chỉ thấy tổng số nguồn, không có danh sách IP. Người tích hợp phải giữ liên hệ counter với receiver/cửa sổ và nguồn lưu lượng, áp policy đã thống nhất. Một quyết định class đơn lẻ không tự cung cấp IP chặn hoặc thời hạn chặn.

11.4. Project Gowin trong gói riêng

`gowin/ai_core_v6.gprj` chỉ dùng Synthesize thành phần AI. Nếu lấy core làm top, giao diện song song có 103 I/O; khi ghép các tín hiệu đó là dây nội bộ của top hệ thống. Không dùng project core để nạp trực tiếp Tang Nano 9K.

Sau khi ghép phải chạy lại mô phỏng hệ thống, P&R, timing và bài kiểm trên bo. Giữ đồng bộ model/RTL/header/vector; không trộn model mới với RTL cũ. V6.1 từng sửa MAC của bản ghép không đồng nghĩa cần học một bộ trọng số AI khác.

12. Đọc mã nguồn theo thứ tự nào? Chạy lệnh nào?

12.1. Bản đồ file

Các đường dẫn dưới đây tính từ gốc từng gói đã bàn giao, không phải một đường dẫn tuyệt đối bắt buộc trên máy bạn.

Trong gói	File nên đọc	Tìm hiểu điều gì?
AI riêng	README.md	Phạm vi bàn giao
AI riêng	spec/FEATURES_V0.md	Chính xác sáu đặc trưng V6; tên v0 được giữ theo chú thích header
AI riêng	model/model.json	Kiến trúc, divisor, shift, weight, bias
AI riêng	tools/ai_reference.py	Phép tính chuẩn số nguyên, ngắn và dễ đọc
AI riêng	firmware/ai_quant_v6.h	Counter -> q -> packing
AI riêng	firmware/ai_v6_integration.h	N = 0 bỏ suy luận
AI riêng	rtl/ai_core_model_v6.v	ROM, signed MAC, state machine, DONE
AI riêng	verification/tb_ai_core.v	Các tình huống giao thức và score/class
AI riêng	model/metrics.json	Các nhóm kết quả, lỗi và giới hạn

Trong gói	File nên đọc	Tìm hiểu điều gì?
Train/dữ liệu	runtime/part2/features.py	PCAP -> thống kê theo timestamp/receiver
Train/dữ liệu	prepare_data.py	Cách tạo profile và gán status
Train/dữ liệu	train.py	Fit/cân bằng/chọn model
Train/dữ liệu	runtime/part2/filtered_calibration.py	Float -> integer và chọn scale
Train/dữ liệu	data/MANIFEST.json, DATA_PROVENANCE.json	Nguồn, biến đổi và checksum

Trong `features.py` có các tùy chọn dùng cho công cụ tổng quát. Hợp đồng V6 đã chốt dùng receiver /32, một giây, exact và divisors cố định; không đổi sang nhánh hash hoặc tự fit divisor rồi mặc định model cũ vẫn đúng.

12.2. Thực hành nhanh với gói học

Tại thư mục chứa tài liệu và `HOC_AI.py`:

```
python3 HOC_AI.py
python3 HOC_AI.py vi-du syn
python3 HOC_AI.py raw 489 40 125 2 1 1
python3 HOC_AI.py counter 489 19728 483 2 1 1
python3 HOC_AI.py raw 803 40 127 1 1 1
python3 HOC_AI.py kiem-tra
```

Windows dùng `python` hoặc `py` thay `python3`. Script chỉ cần Python chuẩn, không cần cài NumPy/sklearn. Hai lệnh raw/counter ở giữa phải cho cùng q, hidden, scores và class. Lệnh 803 phải cho hòa score0/score1 và class 0; đó là tái hiện lỗi, không phải bài kiểm thất bại.

Muốn chạy từ thư mục nào cũng được, kéo file `HOC_AI.py` vào terminal để lấy đường dẫn thật, hoặc mở terminal ngay trong thư mục đã giải nén. Không chép cố định tên tài khoản Admin hoặc đường dẫn của người khác.

12.3. Kiểm bộ bàn giao và tái tạo train

Tại thư mục gói AI riêng:

```
python3 KIEM_TRA_PHAN2.py
python3 KIEM_TRA_PHAN2.py --c --rtl
```

Lệnh đầu chỉ cần Python. Lệnh sau cần GCC và Icarus Verilog, kiểm packing C và core RTL. Thiếu compiler là thiếu công cụ để chạy phần đó, không có nghĩa model đã sai.

Tại thư mục gói train/dữ liệu, dùng Python của venv AI đã cài đúng thư viện:

```
python test_data.py
python evaluate_historical_lab.py
python train.py --output THU_MUC_UNG_VIEN_MOI
```

Đối số output phải là thư mục mới chưa tồn tại. Học cách đọc dữ liệu và chạy suy luận trước; không cần train lại mỗi lần mở bài học. Các lệnh này không tự thay core hoặc firmware hệ thống đã bàn giao.

13. Lộ trình học mười buổi

Mỗi buổi khoảng 60–90 phút, có thể chia nhỏ. Chưa giải thích được đầu ra của buổi thì ôn lại trước khi chuyển tiếp.

Buổi	Học và thực hành	Sản phẩm tự làm
1	Mục 1; phân biệt train/inference và vai trò các phần	Vẽ sơ đồ toàn đường đi, nói vị trí AI trong hai phút
2	Mục 2; xem TCP SYN/SYN-ACK/ACK và cổng nguồn/đích	Giải thích vì sao một SYN không đủ kết luận tấn công
3	Mục 3–4; mở ba CSV mẫu và đọc provenance	Bảng nguồn/nhãn/đơn vị của từng file
4	Mục 5; giải nén và mở ba PCAP trong Wireshark	Ba ảnh chụp lọc cùng receiver/cửa sổ; ghi bằng chứng
5	Mục 6–7; tính raw/q/hex bằng tay rồi chạy script	Bảng tính ba mẫu và một trường hợp N = 0
6	Mục 8; mở model.json và tính neuron/score	Tính đúng h0 của mẫu SYN, giải thích đủ 83 tham số
7	Mục 9; đọc train.py và calibration	Sơ đồ fit -> chọn -> lượng tử hóa -> xuất; chỉ ra chỗ dữ liệu chung nguồn
8	Mục 10; đọc confusion matrix và lỗi SYN 803	Tính precision SYN; ghi đúng ba giới hạn của model
9	Mục 11–12; đọc state machine và kiểm bộ bàn giao	Giải thích handshake, packing score và 84 chu kỳ
10	Mục 14–15; làm bài tập rồi trình bày thử	Bản trình bày 5–7 phút, trả lời câu hỏi không nhìn tài liệu

Không cần thuộc giá trị mọi weight. Cần biết weight ở đâu, số học nào được áp dụng và cách kiểm một phép tính. Đừng lấy số liệu bản V4 làm số liệu V6.

14. Bài tập và đáp án

Hãy tự làm trước khi xem đáp án. Ghi rõ đơn vị, nhãn đang có và phần nào chỉ là dự đoán.

Bài 1 — Nhận dạng file

File A có `id.orig_h, id.resp_h, duration, orig_pkts`; file B có `window_start, packet_count, q0...q5`; file C mở trong Wireshark có 420 frame. Chúng biểu diễn gì? Có thể dùng số dòng A như số packet hay không?

Đáp án: A là flow/connection CSV; B là feature CSV theo receiver/cửa sổ; C là PCAP. Không dùng số flow làm số packet; một flow có thể nhiều packet và trải nhiều cửa sổ.

Bài 2 — Tính hai định dạng đầu vào

Counter là [489,19728,483,2,1,1]. Tính raw và q. Tại sao không truyền [489,40,125,2,1,1] vào hàm C counter?

Đáp án: raw [489,40,125,2,1,1], q [61,20,125,2,1,1]. C cần tổng byte 19728 và số SYN 483; 40/125 là mean và tỷ lệ đã tính, truyền vào sẽ làm tính sai lần nữa.

Bài 3 — Rỗng và lượng tử hóa

N = 0 khác N = 7 như thế nào? N = 8 và N = 15 có q0 gì? N = 1.016 và N = 10.000 có q0 gì?

Đáp án: N = 0 bỏ inference; N = 7 vẫn inference nhưng q0 = 0. Cả 8 và 15 cho q0 = 1. Cả 1.016 và 10.000 cho q0 = 127. Không kiểm cửa sổ rỗng bằng q0.

Bài 4 — Đọc pattern và bằng chứng

Một nguồn gửi 1.000 SYN tới 1.000 cổng trên một host; nguồn khác gửi 1.000 SYN tới cùng cổng 80. Hai hành vi phù hợp với loại nào? Nếu chỉ có một SYN vào cổng 80 thì sao?

Đáp án: Trường hợp đầu phù hợp vertical scan; trường hợp sau có thể phù hợp SYN flood nếu ngữ cảnh tốc độ/mục đích và phản hồi củng cố. Một SYN có thể là mở kết nối hợp lệ; chưa đủ kết luận. Nhận xét hành vi không thay cho nhãn được kiểm chứng hoặc bảo đảm model sẽ phân loại đúng.

Bài 5 — Counter tổng không chỉ ra nguồn tấn công

CSV ghi src_count = 20, dst_count = 1, dst_port_count = 40. Có khẳng định một nguồn vertical scan không?

Đáp án: Không. Có thể nhiều client hợp lệ truy cập dịch vụ khác nhau. Phải xem cổng đích theo từng nguồn và thời gian trong PCAP; core chỉ thấy tổng cardinality.

Bài 6 — Tính model và signed score

Với q SYN ở mục 8, tính h0 và class. Với score [-10,-3,-20], lớp nào thắng? Score -3 có phải confidence -3% không?

Đáp án: a0 = 2113, h0 = 66, ba score [718,1142,-2011], class 1. Trong ví dụ score âm, class 1 thắng vì -3 lớn nhất. Score không phải phần trăm.

Bài 7 — Hòa score và lỗi có thật

Raw [803,40,127,1,1,1] cho q và score nào? Kết quả được đánh giá thế nào nếu nhãn là SYN?

Đáp án: q [100,20,127,1,1,1], score [1413,1413,-3174], class 0 vì ưu tiên chỉ số nhỏ hơn khi hòa. Đây là false negative SYN. Script tái hiện được lỗi không có nghĩa phải sửa nhãn thành 0.

Bài 8 — Precision và recall

Trong nhóm gốc, 21 SYN đều được gọi SYN nhưng 12 Normal cũng bị gọi SYN. Tính precision/recall SYN và giải thích vì sao accuracy gần 100% chưa đủ.

Đáp án: precision 21/(21+12) = 63,64%, recall 21/21 = 100%. Tập có rất nhiều Normal và chỉ 25 cửa sổ attack nên accuracy bị số lượng Normal chi phối. Các con số này vẫn là đánh giá trên dữ liệu đã phát triển, chưa chứng minh tổng quát.

Bài 9 — Tách nguồn và rò rỉ

Bạn nhân bản PCAP thành 100 file, đổi timestamp rồi chia 80/20 để gọi train/test độc lập. Sai ở đâu? Nếu lấy 12.000 gói đầu và 12.000 gói tiếp theo của cùng đợt thì đã độc lập chưa?

Đáp án: Vẫn cùng packet/hành vi/nguồn; số file không tạo độc lập. Chia hai đoạn cùng đợt giảm trùng packet trực tiếp nhưng chưa có nguồn/phiên độc lập, phải mô tả đúng là development cùng nguồn.

Bài 10 — Tích hợp và thời gian

DONE xuất hiện sau 84 chu kỳ ở 27 MHz. Có nghĩa từ gói đầu tiên đến lúc chặn mất 3,11 microsecond không? Khi busy = 1, gửi START mới thì sao?

Đáp án: Không; còn thời gian cửa sổ một giây, trích đếm, bus và policy. 3,11 microsecond chỉ là phép tính độ trễ core theo clock giả định. START lúc busy bị bỏ qua; không có hàng đợi tự động.

Tự đánh giá

Mỗi bài hai điểm: một điểm cho kết quả, một điểm cho giải thích đúng đơn vị/phạm vi. Mục tiêu học là từ 16/20 trở lên và không nhầm ba điểm nền tảng: nhãn khác dự đoán; counter khác raw; kiểm RTL khác kiểm khả năng phát hiện trên nguồn mới. Điểm tự học không phải chứng nhận hệ thống đã đủ điều kiện triển khai.

15. Chuẩn bị trình bày và câu hỏi bảo vệ

15.1. Dàn ý trình bày 5–7 phút

1. Bài toán: phân loại lưu lượng tới receiver thành Normal, SYN flood và vertical scan, cung cấp kết quả cho phần điều khiển.

2. Dữ liệu: Normal/SYN từ phần đã lọc của IoT-23 và scan công khai Chris Sanders; có các profile cùng nguồn, chưa có kiểm độc lập mới.

3. Đặc trưng: sáu thống kê một giây; mô tả tử số/mẫu số SYN và cổng đích.

4. Model: MLP 6–8–3, một lớp ẩn ReLU, 83 tham số, inference số nguyên.

5. Một ví dụ: counter SYN -> q -> score -> class, dùng đúng các số ở mục 7.

6. Kiểm chứng: C packing, 3.762 vector RTL, 84 chu kỳ; trình bày precision/recall và các lỗi còn lại.

7. Ghép nối: core nhận feat, trả class/scores; CPU/bộ trích đếm/policy do hệ thống kết nối theo hợp đồng.

15.2. Câu hỏi thường gặp và ý trả lời

Câu hỏi	Ý trả lời ngắn cần hiểu
Vì sao không dùng trực tiếp PCAP để train?	MLP này nhận sáu thống kê cố định, không nhận chuỗi bytes có độ dài biến đổi. Cần trích đúng phạm vi trước.
Vì sao chọn sáu đặc trưng này?	Mô tả tải, kích thước gói, tỷ lệ SYN và độ phân tán nguồn/đích/cổng với chi phí tính nhỏ; có giới hạn do tổng hợp. Không khẳng định tối ưu.
Vì sao cần cả SYN ratio và dst_port_count?	Flood và scan đều có thể nhiều SYN; phân tán cổng là thông tin bổ sung, vẫn cần ngữ cảnh.
Vì sao dst_count thường là 1?	Mẫu được định nghĩa theo một receiver /32; hiện tại đây gần như hằng số, giữ để khớp giao diện sáu đầu vào.
6–8–3 có phải ba lớp tấn công không?	Là số đầu vào/neuron ẩn/đầu ra; ba class gồm cả Normal.
Vì sao tám neuron?	Thiết kế nhỏ đã chọn để phù hợp core chia sẻ MAC; chưa chứng minh đây là kiến trúc tối ưu toàn cục.
FPGA có tự train không?	Không; train ở Python, bo inference bằng trọng số nhúng.
Lượng tử hóa và chuẩn hóa có giống nhau không?	Có liên quan scale nhưng là hai bước khác: raw -> q và chuyển phép tính/weight float sang số nguyên.
Vì sao cần int32 accumulator?	Tổng nhiều tích signed và bias lớn hơn phạm vi int8; phải tránh tràn trước khi shift/clip.
Có bao nhiêu tham số?	48+8+24+3 = 83, không tính divisors như weight được fit.
Tại sao accuracy đẹp mà precision SYN chỉ 63,64%?	Có 12 Normal báo nhầm trong 33 dự đoán SYN, còn accuracy bị hàng triệu Normal chi phối.
120 profile có phải 120 phiên attack?	Không; biến thể cùng nguồn.
3.762 vector có phải test mạng độc lập?	Không; kiểm score/class và giao thức core.
Có tự động chặn theo class được không?	Core không có IP; phần điều khiển phải gắn kết quả với nguồn và policy, kiểm toàn hệ thống.
Có đảm bảo mọi scan chậm được nhận ra không?	Không; cửa sổ một giây có thể chỉ vài probe, đặc trưng không đủ ngữ cảnh và nguồn còn ít.
Bản này còn lỗi gì?	12 Normal gốc báo SYN, một SYN 803 gói ở development bị bỏ sót, một Normal lab cũ nhận scan; chưa có test độc lập mới.

15.3. Đoạn giới thiệu bạn có thể tập nói

“Phần của em xây dựng khối AI phân loại lưu lượng theo cửa sổ một giây tại một receiver. Em dùng dữ liệu Normal và SYN đã lọc từ IoT-23 cùng mẫu vertical scan công khai đã rà soát. Sáu đặc trưng được lượng tử hóa thành đầu vào cho MLP 6–8–3; trọng số số nguyên được nhúng trong core Verilog. Em kiểm tính nhất quán Python/C/RTL và giao thức core. Các biến thể mô phỏng vẫn chung nguồn nên em trình bày kết quả là độ khớp và hồi quy hiện có, đồng thời nêu rõ các lỗi và phần kiểm độc lập còn thiếu.”

Hãy nói bằng lời của bạn và chỉ nhận những gì bạn đã hiểu, tự chạy hoặc có bằng chứng. Khi được hỏi ngoài phạm vi, nêu rõ chỗ cần đọc/kiểm thêm thay vì suy diễn từ accuracy.

16. Từ điển ngắn và nguồn đối chiếu

Từ	Nghĩa trong phần AI này
Packet/frame	Gói IP / khung lớp liên kết; hai đơn vị liên quan nhưng không thay tùy tiện độ dài
Flow	Nhóm gói theo kết nối/5-tuple và quy tắc công cụ ghi flow
Window	Một khoảng một giây của receiver được chọn
Feature	Giá trị thống kê đưa vào model
Raw	Sáu đặc trưng trước chia divisor/clip
Counter	Các tổng đếm gốc; B và S trước khi tính mean/ratio
Quantization	Biểu diễn và phép tính bằng số nguyên hữu hạn, có scale/làm tròn/bão hòa
Cardinality	Số giá trị phân biệt, ví dụ số IP nguồn khác nhau
Label	Nhãn tham chiếu của dữ liệu/kịch bản
Prediction	Lớp model suy ra từ đầu vào
Provenance	Nguồn gốc, phép biến đổi, phạm vi và checksum của dữ liệu
Ground truth	Nhãn tham chiếu được kiểm chứng; không phải mọi nhãn file đều chắc chắn tuyệt đối
Overfitting	Hợp dữ liệu đã xem nhưng không giữ chất lượng trên dữ liệu mới
Leakage	Thông tin không nên có lọt vào train hoặc bước lựa chọn
False positive	Bình thường bị báo là tấn công trong bài toán đang xét
False negative	Tấn công bị bỏ sót; trong multiclass phải nói rõ đang xét lớp nào
MAC trong mạng	Media Access Control, lớp giao tiếp Ethernet
MAC trong neuron	Multiply–Accumulate, phép nhân–cộng; khác MAC Ethernet
Synthesis / P&R	Ánh xạ logic / đặt và nối tài nguyên FPGA
STA / CDC	Phân tích timing tĩnh / xử lý giao tiếp giữa miền clock

Nguồn nội bộ để đối chiếu số liệu:

• `model/model.json`, `model/metrics.json`, `model/DATA_PROVENANCE.json` trong gói AI riêng, model hash đã nêu ở đầu.

• `data/MANIFEST.json`, `prepare_data.py`, `train.py`, `runtime/part2/features.py`, `runtime/part2/filtered_calibration.py` và `historical_lab_regression.json` trong gói train/dữ liệu.

• `verification/recorded/check_result.json` và testbench trong gói AI riêng.

• `VI_DU_DA_DOI_CHIEU.json` trong gói học: nguồn, checksum, counter trích lại, hidden và score của ba ví dụ.

Nguồn công khai gốc, không thay cho manifest của phần đã lọc:

• IoT-23: https://www.stratosphereips.org/datasets-iot23

• Bộ packet của Chris Sanders: https://github.com/chrissanders/packets

• Bản scan dùng để chuyển đổi: https://raw.githubusercontent.com/chrissanders/packets/0412fb1dc0aa671128f9da6689e0dbd4459c0b51/synscan.pcapng

Trong gói học, nhãn scan là kết quả rà soát hành vi của dự án; PCAP mô phỏng là profile dẫn xuất, không gán cho tác giả gốc các biến thể hoặc nhãn do nhóm tạo. Các phép tính mẫu được chạy bằng đúng model V6 và đối chiếu lại với ba PCAP khi soạn tài liệu.